LAPORAN PRAKTIKUM DEVSECOPS
KELOMPOK 3 - PMB DevSecOps Lab 6

Nama	: Muhammad Rafi Putra Hidayat
	Hafidz Kurnia Darojad
	Indra Rahmat Maulidi
	Moh Iqbal Triwijaya
NRP	: 3124600009
	3124600015
	3124600017
	3126640015
Dosen Pengajar	: Ferry Astika Saputra, ST., M.Sc

PROGRAM STUDI D4 TEKNIK INFORMATIKA

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA (PENS)

TAHUN 2026

Quick start

```
cd pmb-devsecops-lab
./scripts/setup.sh
./scripts/validate.sh
docker compose up -d --build
```

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ ./scripts/setup.sh
[PASS] .env dibuat dengan credential acak.
[INFO] Login laboratorium: student / StudentLab!2026
[INFO] Berikutnya: make validate && make up
```

Alur kerja scripts/setup.sh:

1. Set direktori: Pindah working directory ke root project.
2. Generasi .env:
    a. Cek file .env. Jika sudah ada, pertahankan.
    b. Jika belum ada: salin dari .env.example, set izin 600 (hanya owner).
    c. Generate string acak 48-hex via openssl / python3 secrets untuk 5 secret:
    d. APP_DB_ADMIN_PASSWORD
    e. APP_DB_WRITER_PASSWORD
    f. LOG_DB_ADMIN_PASSWORD
    g. LOG_DB_WRITER_PASSWORD
    h. SESSION_SECRET
3. Set permission script: Jalankan chmod +x untuk init-app/01-init-app.sh, init-log/01-init-log.sh, dan seluruh file scripts/*.sh.
4. Output CLI: Tampilkan credential lab default (student / StudentLab!2026) dan perintah langkah berikutnya (make validate && make up).

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ ./scripts/validate.sh
Can't list 'tests'
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
ModuleNotFoundError: No module named 'yaml'
```

1. Can't list 'tests': Python compileall cari direktori tests/ di root. Tidak ada (lokasi asli di app/tests/). Kompilasi fallback ke app/. Normal, bukan error fatal.
2. ModuleNotFoundError: No module named 'yaml': Script validasi butuh library PyYAML untuk parse compose.yaml. Python sistem belum ada modul tersebut. Eksekusi stop.

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ sudo apt install python3.13-venv
Installing:
  python3.13-venv

Installing dependencies:
  python3-pip-whl  python3-setuptools-whl

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 0
  Download size: 2,781 kB
  Space needed: 2,951 kB / 10.3 GB available
```

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ python3 -m venv .venv
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ source .venv/bin/activate
(.venv) user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ pip install pyyaml
WARNING: The directory '/data/home/.cache/pip' or its parent directory is not own
o's -H flag.
Collecting pyyaml
  Downloading pyyaml-6.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64
Downloading pyyaml-6.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.m
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 801.6/801.6 kB 51.5 MB/s eta 0:00:00
Installing collected packages: pyyaml
Successfully installed pyyaml-6.0.3
(.venv) user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$
```

```
(.venv) user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ ./scripts/validate.sh
Can't list 'tests'
[PASS] YAML Compose dapat diparse dan enam service ditemukan
test_invalid_request_id_is_replaced (test_common.CommonTests.test_invalid_request_id_is_replaced) ... ok
test_sensitive_fields_are_redacted (test_common.CommonTests.test_sensitive_fields_are_redacted) ... ok
test_valid_request_id_is_preserved (test_common.CommonTests.test_valid_request_id_is_preserved) ... ok

----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
[PASS] docker compose config
[PASS] Validasi statis selesai.
```

1. Install python3.13-venv, buat virtual environment .venv, aktifkan, install pyyaml via pip.

Alasan memakai .venv:

1. Proteksi OS (PEP 668): Linux modern (Ubuntu/Debian) blokir pip install di level sistem (externally-managed-environment). Mencegah file apt rusak tertimpa library Python manual.
2. Isolasi dependensi: Library (pyyaml, dll.) terisolasi di direktori project. Hindari konflik versi dengan script bawaan OS atau project lain.

3. Keamanan permission: Tanpa hak root/sudo. Modifikasi library terbatas dalam folder .venv.

```
(.venv) user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ docker compose up -d --build
[+] up 39/39
 ✔ Image postgres:16-alpine      Pulled
 ✔ Image fluent/fluent-bit:4.2.8 Pulled
[+] Building 6.3s (17/17) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 1.49kB
```

```
(.venv) user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:8080/healthz
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 0 ms: Could not connect to server
(.venv) user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:2020/api/v1/health
curl: (7) Failed to connect to 127.0.0.1 port 2020 after 1 ms: Could not connect to server
```

Tunggu hingga service sehat:

```
docker compose ps
curl -fsS http://127.0.0.1:8080/healthz
curl -fsS http://127.0.0.1:2020/api/v1/health
```

```
(.venv) user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ docker compose ps
NAME                          IMAGE                          COMMAND                  SERVICE              CREATED              STATUS                        PORTS
pmb-lab-app-db-1              postgres:16-alpine             "docker-entrypoint.s…"   app-db               About a minute ago   Up About a minute (healthy)   5432/tcp
pmb-lab-auth-service-1        pmb-lab-auth-service           "gunicorn --bind 0.0…"   auth-service         About a minute ago   Up 55 seconds (healthy)       8080-8081/tcp
pmb-lab-fluent-bit-1          fluent/fluent-bit:4.2.8        "/fluent-bit/bin/flu…"   fluent-bit           About a minute ago   Up 55 seconds                 2020/tcp, 24224/tcp
pmb-lab-log-db-1              postgres:16-alpine             "docker-entrypoint.s…"   log-db               About a minute ago   Up About a minute (healthy)   5432/tcp
pmb-lab-registration-api-1    pmb-lab-registration-api       "gunicorn --bind 0.0…"   registration-api     About a minute ago   Up 49 seconds (healthy)       8080-8081/tcp
pmb-lab-verification-worker-1 pmb-lab-verification-worker    "python worker.py"       verification-worker  About a minute ago   Up 54 seconds                 8080-8081/tcp
```

```
(.venv) user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:8080/healthz
{"service":"registration-api","status":"ok"}
(.venv) user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:2020/api/v1/health

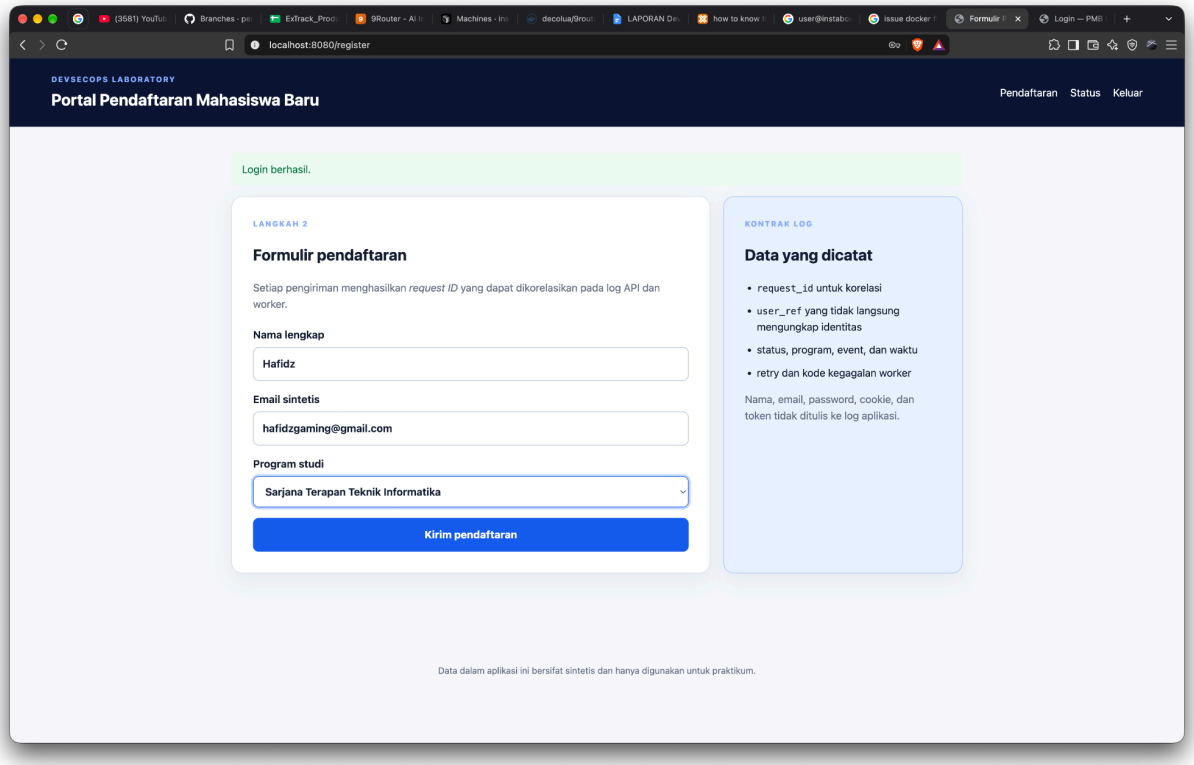
{"fluent-bit":{"version":"4.2.8","edition":"Community","flags":["FLB_HAVE_SYS_WAIT_H","FLB_HAVE_SIMD","FLB_HAVE
DER","FLB_EVENT_LOOP_AUTO_DISCOVERY","FLB_HAVE_TLS","FLB_HAVE_OPENSSL","FLB_HAVE_METRICS","FLB_HAVE_PROFILES","
FLB_HAVE_SIGNV4","FLB_HAVE_SQLDB","FLB_LOG_NO_CONTROL_CHARS","FLB_HAVE_METRICS","FLB_HAVE_HTTP_SERVER","FLB_HAV
YSTEM","FLB_HAVE_LIBYAML","FLB_HAVE_ATTRIBUTE_ALLOC_SIZE","FLB_HAVE_PROXY_GO","FLB_HAVE_JEMALLOC","FLB_HAVE_LIB
_GETENTROPY_SYS_RANDOM"]}}(.venv) user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$
```

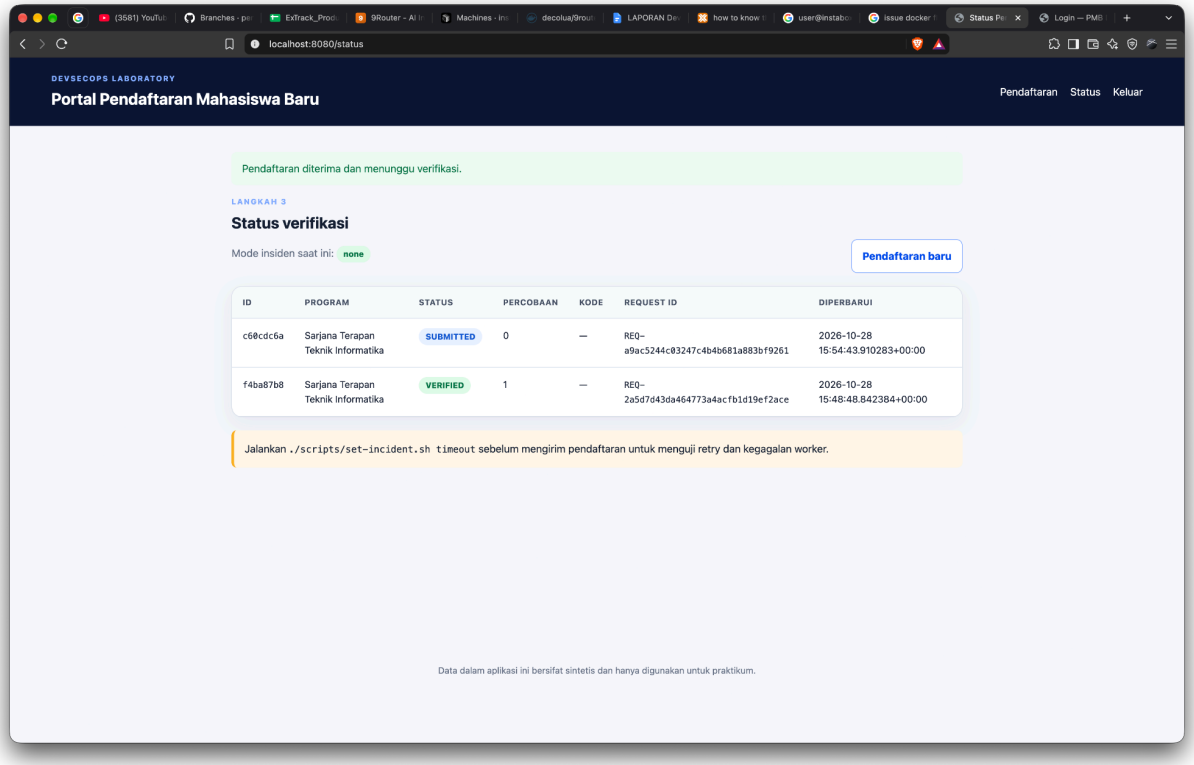
Buka http://127.0.0.1:8080 pada browser.

Username : student

Password : StudentLab!2026

setup.sh membuat .env dengan credential database dan session secret acak. Jangan memasukkan .env ke repository atau laporan.

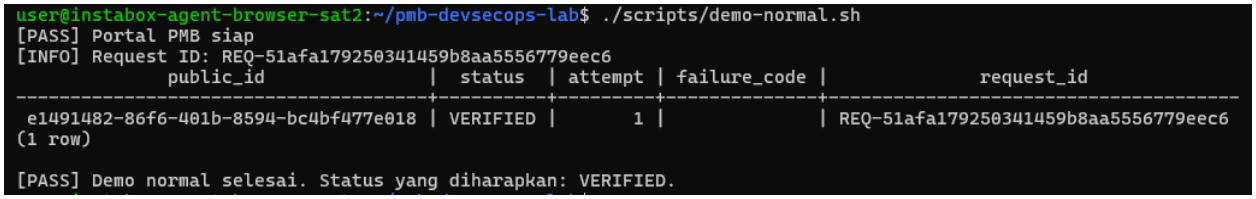

localhost:8080/register
DEVSECOPS LABORATORY
Portal Pendaftaran Mahasiswa Baru
Pendaftaran
Status
Keluar
Login berhasil.
LANGKAH 2
Formulir pendaftaran
Setiap pengiriman menghasilkan request ID yang dapat dikorelasikan pada log API dan worker.
Nama lengkap
Hafidz
Email sintetis
hafidzgaming@gmail.com
Program studi
Sarjana Terapan Teknik Informatika
Kirim pendaftaran
KONTRAK LOG
Data yang dicatat
request_id untuk korelasi
user_ref yang tidak langsung mengungkap identitas
status, program, event, dan waktu
retry dan kode kegagalan worker
Nama, email, password, cookie, dan token tidak ditulis ke log aplikasi.
Data dalam aplikasi ini bersifat sintetis dan hanya digunakan untuk praktikum.


localhost:8080/status
DEVSECOPS LABORATORY
Portal Pendaftaran Mahasiswa Baru
Pendaftaran
Status
Keluar
Pendaftaran diterima dan menunggu verifikasi.
LANGKAH 3
Status verifikasi
Mode insiden saat ini: none
Pendaftaran baru
ID
PROGRAM
STATUS
PERCOBAAN
KODE
REQUEST ID
DIPERBARUI
c60cdc6a
Sarjana Terapan Teknik Informatika
SUBMITTED
0
—
REQ-a9ac5244c03247c4b4b681a883bf9261
2026-10-28 15:54:43.910283+00:00
f4ba87b8
Sarjana Terapan Teknik Informatika
VERIFIED
1
—
REQ-2a5d7d43da464773a4acfb1d19ef2ace
2026-10-28 15:48:48.842384+00:00
Jalankan ./scripts/set-incident.sh timeout sebelum mengirim pendaftaran untuk menguji retry dan kegagalan worker.
Data dalam aplikasi ini bersifat sintetis dan hanya digunakan untuk praktikum.

Skenario 1 — Alur normal

Jalankan otomatis:

`./scripts/demo-normal.sh`



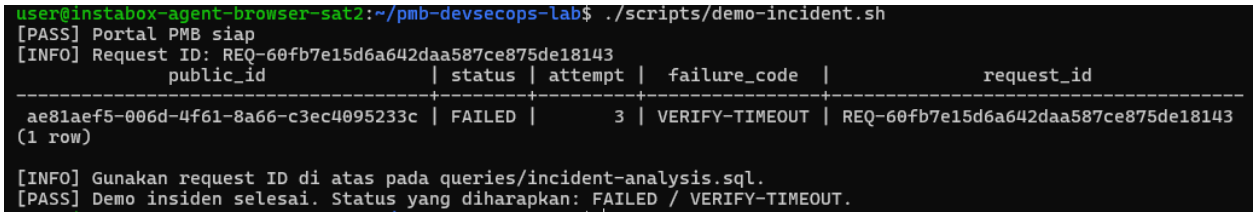
Alur kerja scripts/demo-normal.sh:

- `set -Eeuo pipefail, cd, load_env` : mengatur mode penanganan error, berpindah ke root project serta menjalankan scripts/lib.sh dan .env
- `wait_http` : menunggu hingga service berjalan normal
- `set-incident.sh` : Mengirimkan data pendaftaran baru dengan tipe NORMAL dan mengambil serta menyimpan request_id yang dihasilkan dari respons tersebut
- `docker compose exec ... psql` : Menjalankan query SQL langsung ke database PostgreSQL (app-db) untuk memeriksa status data pendaftaran berdasarkan request_id yang baru saja dibuat. Data yang ditampilkan meliputi public_id, status, attempt, failure_code, dan request_id

Skenario 2 — Timeout dependency

Jalankan otomatis:

`./scripts/demo-incident.sh`



Alur kerja scripts/demo-incident.sh:

- `set-incident.sh timeout` : Mengubah mode simulasi gangguan pada sistem menjadi `timeout`. Hal ini akan membuat proses verifikasi pendaftaran mengalami kendala atau waktu habis (*timeout*) pada dependensi eksternal/worker.
- `submit_application INCIDENT` : Mengirimkan data pendaftaran baru dengan tipe INCIDENT dan menangkap request_id yang dihasilkan dari transaksi tersebut
- `docker compose exec ... psql` : menjalankan query SQL yang mirip dengan demo normal
- Memberikan informasi untuk menggunakan request_id tsb

Menelusuri request ID

Setiap respons aplikasi membawa header X-Request-ID. Nilainya juga disimpan pada transaksi dan event log.

Tampilkan transaksi:

```
set -a; source .env; set +a
docker compose exec -T app-db \
  psql -U "$APP_DB_ADMIN_USER" -d "$APP_DB_NAME" \
  -c 'SELECT public_id, status, attempt, failure_code, request_id
FROM registrations ORDER BY submitted_at DESC;'
```

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ set -a; source .env; set +a
docker compose exec -T app-db \
  psql -U "$APP_DB_ADMIN_USER" -d "$APP_DB_NAME" \
  -c 'SELECT public_id, status, attempt, failure_code, request_id FROM registrations ORDER BY submitted_at DESC;'
             public_id              |  status  | attempt |  failure_code  |              request_id
------------------------------------+----------+---------+----------------+------------------------------------
 ae81aef5-006d-4f61-8a66-c3ec4095233c | FAILED   |       3 | VERIFY-TIMEOUT | REQ-60fb7e15d6a642daa587ce875de18143
 e1491482-86f6-401b-8594-bc4bf477e018 | VERIFIED |       1 |                | REQ-51afa179250341459b8aa5556779eec6
 c60cdc6a-70b8-4dc3-b050-5be16c73a038 | VERIFIED |       1 |                | REQ-a9ac5244c03247c4b4b681a883bf9261
 f4ba87b8-c663-48e0-b318-daa671e1355f | VERIFIED |       1 |                | REQ-2a5d7d43da464773a4acfb1d19ef2ace
(4 rows)
```

Salin salah satu request ID, kemudian ubah variabel `investigated_request_id` pada `queries/incident-analysis.sql` dan jalankan:

```
-- 4. Ganti nilai request ID untuk merekonstruksi satu transaksi.
\set investigated_request_id 'REQ-60fb7e15d6a642daa587ce875de18143'
WITH events AS (
    SELECT time, tag, (data ->> 'log')::jsonb AS event
    FROM logs.fluentbit
    WHERE tag LIKE 'pmb.%'
      AND left(ltrim(data ->> 'log'), 1) = '{'
)
```

```
docker compose exec -T log-db \
  psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" \
  --file=/dev/stdin < queries/incident-analysis.sql
```

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ docker compose exec -T log-db \
  psql -U "$LOG_DB_ADMIN_USER" -d "$LOG_DB_NAME" \
  --file=/dev/stdin < queries/incident-analysis.sql
Pager usage is off.
Timing is on.
              tag               | total_event
--------------------------------+-------------
 pmb.pmb-lab-registration-api-1 |        1444
 pmb.pmb-lab-auth-service-1     |         702
 pmb.pmb-lab-verification-worker-1 |          12
(3 rows)

Time: 2.575 ms
```

```
        time         |     service      | severity |  event_name  |             request_id             | registration_id | error_code
---------------------+------------------+----------+--------------+------------------------------------+-----------------+------------
 2026-10-28 16:47:33 | registration-api | INFO     | http_request | REQ-78570f8a2d7a443d8fbbc22ce0ea91a3 |                 |
 2026-10-28 16:47:28 | registration-api | INFO     | http_request | REQ-1d8da375752d4b178b90fba74d3bdac8 |                 |
 2026-10-28 16:47:23 | registration-api | INFO     | http_request | REQ-445bc1e8f69f446ca1db3e431e6e5fee |                 |
 2026-10-28 16:47:18 | registration-api | INFO     | http_request | REQ-756fd188e80541dca091ca254e5af030 |                 |
 2026-10-28 16:47:13 | registration-api | INFO     | http_request | REQ-44868406e8fa4156932082676b3bc312 |                 |
 2026-10-28 16:47:07 | registration-api | INFO     | http_request | REQ-0be4fcedbeaa416684226e975b298f8b |                 |
 2026-10-28 16:47:02 | registration-api | INFO     | http_request | REQ-5fef27acdde14d5bac24a18d669f9ced |                 |
 2026-10-28 16:46:57 | registration-api | INFO     | http_request | REQ-197cfc4bddf84a358b9878767e6059d6 |                 |
 2026-10-28 16:46:52 | registration-api | INFO     | http_request | REQ-851e6078c9b64a0893b6d72f14c1c0c8 |                 |
 2026-10-28 16:46:47 | registration-api | INFO     | http_request | REQ-9f0a7b81f1744335b8652e0df571ff9e |                 |
 2026-10-28 16:46:42 | registration-api | INFO     | http_request | REQ-e311a350031a4ba8ad15b9874df57375 |                 |
 2026-10-28 16:46:36 | registration-api | INFO     | http_request | REQ-95ff757b700648769b8a2870dfddb30a |                 |
 2026-10-28 16:46:31 | registration-api | INFO     | http_request | REQ-47ee06e74b6c4879ac035e978d5f729d |                 |
 2026-10-28 16:46:26 | registration-api | INFO     | http_request | REQ-71779dcdc1e0451db61c4813f7d9cce8 |                 |
 2026-10-28 16:46:21 | registration-api | INFO     | http_request | REQ-e600e34d169e49a08a846d3eafa9a1cf |                 |
 2026-10-28 16:46:16 | registration-api | INFO     | http_request | REQ-db2413d80a834cbaa5a9b662a53d913e |                 |
 2026-10-28 16:46:10 | registration-api | INFO     | http_request | REQ-0a32d140a6cb4ec7b268d84459bf7081 |                 |
 2026-10-28 16:46:05 | registration-api | INFO     | http_request | REQ-dc210d3416d64d058d0b297c02eba018 |                 |
 2026-10-28 16:46:00 | registration-api | INFO     | http_request | REQ-cf939ff6ebfa47c8823b51a6bec15999 |                 |
```

```
 2026-10-28 16:43:45 | registration-api | INFO     | http_request | REQ-e3a35adf70b24b94b2574f515c7e10e9 |                 |
 2026-10-28 16:43:40 | registration-api | INFO     | http_request | REQ-ffee598414164638b40a3c215d5507f2 |                 |
 2026-10-28 16:43:35 | registration-api | INFO     | http_request | REQ-f65c9dd4dfc8418c85eab516aaffb13d |                 |
 2026-10-28 16:43:30 | registration-api | INFO     | http_request | REQ-b7efa7ac2f5f4df79c107c2298725177 |                 |
 2026-10-28 16:43:25 | registration-api | INFO     | http_request | REQ-2f3f5b07e17741d3a014fbd74f3f799a |                 |
 2026-10-28 16:43:20 | registration-api | INFO     | http_request | REQ-63d15a756649474a94078ebc7d826802 |                 |
(50 rows)

Time: 9.204 ms
       service       | severity | total
---------------------+----------+-------
 verification-worker | WARN     |     3
 registration-api    | WARN     |     1
 verification-worker | ERROR    |     1
(3 rows)

Time: 2.211 ms
        time         |       service       |       event_name       | severity | attempt |   error_code
---------------------+---------------------+------------------------+----------+---------+----------------
 2026-10-28 16:33:34 | registration-api    | registration_submitted | INFO     |         |
 2026-10-28 16:33:34 | registration-api    | http_request           | INFO     |         |
 2026-10-28 16:33:36 | verification-worker | verification_started   | INFO     |         |
 2026-10-28 16:33:36 | verification-worker | verification_retry     | WARN     | 1       |
 2026-10-28 16:33:37 | verification-worker | verification_retry     | WARN     | 2       |
 2026-10-28 16:33:38 | verification-worker | verification_retry     | WARN     | 3       |
 2026-10-28 16:33:39 | verification-worker | dependency_timeout     | ERROR    | 3       | VERIFY-TIMEOUT
(7 rows)

Time: 2.158 ms
    minute_bucket    | total
---------------------+-------
 2026-10-28 16:47:00 |    21
 2026-10-28 16:46:00 |    36
 2026-10-28 16:45:00 |    33
 2026-10-28 16:44:00 |    36
 2026-10-28 16:43:00 |    34
 2026-10-28 16:42:00 |    35
 2026-10-28 16:41:00 |    36
 2026-10-28 16:40:00 |    33
 2026-10-28 16:39:00 |    36
 2026-10-28 16:38:00 |    34
 2026-10-28 16:37:00 |    35
 2026-10-28 16:36:00 |    34
 2026-10-28 16:35:00 |    35
 2026-10-28 16:34:00 |    36
 2026-10-28 16:33:00 |    51
(15 rows)

Time: 0.519 ms
 time | tag | suspicious_message
------+-----+--------------------
(0 rows)

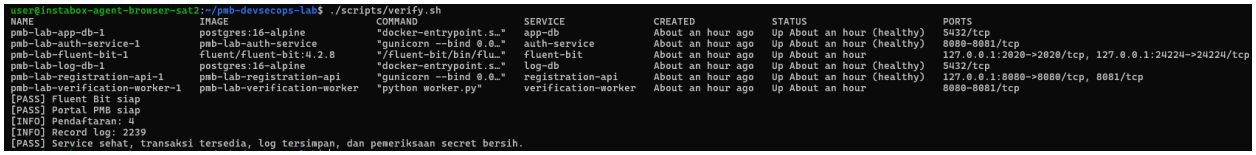
Time: 17.161 ms
 candidate_for_deletion
------------------------
                      0
(1 row)

Time: 0.319 ms
```

Pengujian pipeline

Verifikasi umum:

```
./scripts/verify.sh
```

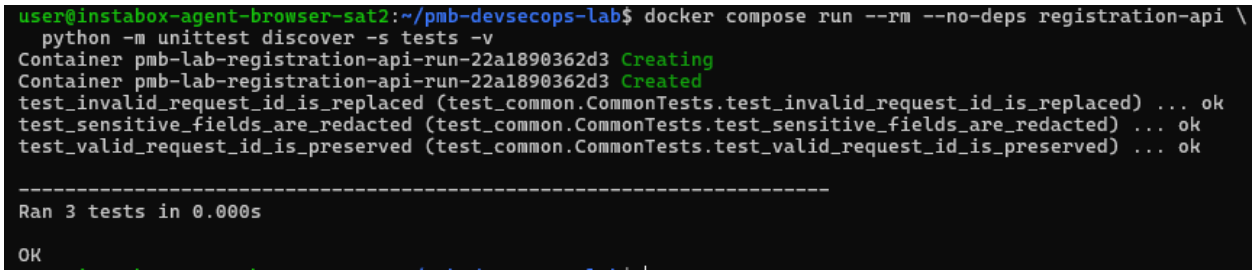


Alur kerja scripts/verify.sh:

- `app_rows`, `log_rows` :
  - Menghitung jumlah total data pendaftaran di database aplikasi (`app-db`).
  - Menghitung jumlah total log terpusat yang masuk ke database log (`log-db`) melalui Fluent Bit.
  - Jika jumlah transaksi atau log kurang dari 1, script akan menghentikan proses (`exit 1`) dan memberikan peringatan untuk menjalankan demo normal/insiden terlebih dahulu.
- `suspicious` :
  - Menjalankan kueri SQL pada penyimpanan log terpusat untuk mendeteksi apakah ada informasi sensitif atau rahasia yang tidak sengaja tercatat di dalam log (seperti `password`, `authorization`, `bearer`, `cookie`, `session_secret`, atau `api_key`).
  - Jika ditemukan adanya data sensitif (`suspicious > 0`), script akan menggagalkan verifikasi untuk mencegah kebocoran informasi.
- Jika semua pemeriksaan berhasil, script akan menampilkan pesan sukses

Unit test redaction dan request ID:

```
docker compose run --rm --no-deps registration-api \
  python -m unittest discover -s tests -v
```



digunakan untuk menjalankan unit test secara otomatis di dalam container layanan registration-api tanpa memuat container dependensinya (--no-deps)

Evidence

`./scripts/collect-evidence.sh`

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ ./scripts/collect-evidence.sh
[PASS] Evidence dibuat di direktori reports/. File .env tidak disertakan.
```

Alur kerja `./scripts/collect-evidence.sh` :

- Membuat folder reports/ jika belum ada untuk menyimpan seluruh file bukti.
- Menyimpan daftar dan status berjalan seluruh kontainer Docker Compose ke dalam file reports/compose-ps.txt
- Mengambil data metrik performa/layanan dari API Fluent Bit (http://127.0.0.1:2020/api/v1/metrics) dan menyimpannya dalam format JSON di reports/fluent-bit-metrics.json
- Menjalankan kueri SQL pada database aplikasi (app-db) untuk mengekstrak seluruh data pendaftaran (public_id, status, attempt, failure_code, request_id, updated_at) yang diurutkan berdasarkan waktu kirim, lalu menyimpannya ke reports/registration-status.txt
- Menjalankan perintah/kueri SQL pada database log terpusat (log-db) melalui standard input (stdin) untuk memproses atau menyimpan data pencatatan log.

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab$ tree reports/
reports/
├── compose-ps.txt
├── config-sha256.txt
├── fluent-bit-metrics.json
├── incident-analysis.txt
└── registration-status.txt

1 directory, 5 files
```

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab/reports$ cat compose-ps.txt
NAME                          IMAGE                          COMMAND                  SERVICE             CREATED             STATUS                        PORTS
pmb-lab-app-db-1              postgres:16-alpine             "docker-entrypoint.s…"   app-db              About an hour ago   Up About an hour (healthy)    5432/tcp
pmb-lab-auth-service-1        pmb-lab-auth-service           "gunicorn --bind 0.0…"   auth-service        About an hour ago   Up About an hour (healthy)    8080-8081/tcp
pmb-lab-fluent-bit-1          fluent/fluent-bit:4.2.8        "/fluent-bit/bin/flu…"   fluent-bit          About an hour ago   Up About an hour              127.0.0.1:2020->2020/tcp, 127.0.0.1:24224->24224/tcp
pmb-lab-log-db-1              postgres:16-alpine             "docker-entrypoint.s…"   log-db              About an hour ago   Up About an hour (healthy)    5432/tcp
pmb-lab-registration-api-1    pmb-lab-registration-api       "gunicorn --bind 0.0…"   registration-api    About an hour ago   Up About an hour (healthy)    127.0.0.1:8080->8080/tcp, 8081/tcp
pmb-lab-verification-worker-1 pmb-lab-verification-worker    "python worker.py"       verification-worker About an hour ago   Up About an hour              8080-8081/tcp
```

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab/reports$ cat config-sha256.txt
defc9e02919ec95b1250e7c6055076df933cd2270f2e7fd9a0c4d39aa16bded2  compose.yaml
3208265779fedcc782027c70fcad910910f4aa8f77774e12852f8464aa053929  app/auth_service.py
6a8ebd2bfecb43939564724c0a5493f55d263039f4e2622bd7761515edb0a554  app/common.py
1286e2bd630279fb7d16876520e5ceda84220649c9e076097f201608711f129c  app/registration_api.py
f202d1ff1dc32e69b6a857692326b38d4f3f44c9ab2636ea65a190f185dc3dde  app/worker.py
c18a8d93144e96a1bda9804c552a5f8b0ab0e7adcca7d80e42803c0d977506f9  app/requirements.txt
775cda790f80e7d3c7c48f901567cae2374fb1bc1996ed5145b3febff4f11dc0  fluent-bit/fluent-bit.conf
182c9557c055da6321080ba54940829331febd6e130045aae2ff7b1ac1fe93c1  init-app/01-init-app.sh
4614c85f4a4eae7e4bfa9ad1b0c019a17465dd206d6908f8036bc50c9bb28221  init-log/01-init-log.sh
91bbfc3c877293a65ae77d9a27599190f08beed1252090c89fba56205a79a725  queries/incident-analysis.sql
```

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab/reports$ cat fluent-bit-metrics.json
{"input":{"forward.0":{"records":2468,"bytes":747162},"storage_backlog.1":{"records":0,"bytes":0}},"filter":{},"output":{"pgsql.0":{"proc_records":2468,"proc_bytes":74716
2,"errors":0,"retries":0,"retries_failed":0,"dropped_records":0,"retried_records":0},"stdout.1":{"proc_records":2468,"proc_bytes":747162,"errors":0,"retries":0,"retries_f
ailed":0,"dropped_records":0,"retried_records":0}}}user@instabox-agent-browser-sat2:~/pmb-devsecops-lab/reports$ cat incident-anal
```

```
user@instabox-agent-browser-sat2:~/pmb-devsecops-lab/reports$ cat registration-status.txt
              public_id               |  status  | attempt |  failure_code  |              request_id              |          updated_at
--------------------------------------+----------+---------+----------------+--------------------------------------+-------------------------------
 f4ba87b8-c663-48e0-b318-daa671e1355f | VERIFIED |       1 |                | REQ-2a5d7d43da464773a4acfb1d19ef2ace | 2026-10-28 15:48:48.842384+00
 c60cdc6a-70b8-4dc3-b050-5be16c73a038 | VERIFIED |       1 |                | REQ-a9ac5244c03247c4b4b681a883bf9261 | 2026-10-28 15:54:45.335321+00
 e1491482-86f6-401b-8594-bc4bf477e018 | VERIFIED |       1 |                | REQ-51afa179250341459b8aa5556779eec6 | 2026-10-28 16:32:01.63228+00
 ae81aef5-006d-4f61-8a66-c3ec4095233c | FAILED   |       3 | VERIFY-TIMEOUT | REQ-60fb7e15d6a642daa587ce875de18143 | 2026-10-28 16:33:39.060043+00
(4 rows)
```

Pertanyaan

Kapan kita memakai fluentbit, kapan kita memakai prometheus?

Perbedaan utama ada pada jenis data telemetri: Fluent Bit untuk LOGS (kejadian diskrit/teks), Prometheus untuk METRICS (angka deret waktu/time-series).

GUNAKAN FLUENT BIT (Bab 6 — Centralized Logging) SAAT:

1. Mengumpulkan event diskrit per kejadian (baris stdout/stderr container, error stack trace, audit trail login, request detail).
2. Butuh konteks kardinalitas tinggi (user_id, request_id, IP, pesan error spesifik, payload JSON).
3. Container bersifat ephemeral dan log perlu diparsing, difilter, lalu dikirim (push/forward) ke penyimpanan terpusat seperti PostgreSQL JSONB, Loki, atau Elasticsearch untuk forensik dan debugging.
4. Menjawab pertanyaan: "Mengapa error ini terjadi dan pesan/baris kode mana penyebabnya?"

GUNAKAN PROMETHEUS (Bab 7 — Monitoring Metrics) SAAT:

1. Mengukur kondisi numerik sistem secara periodik (CPU, RAM, disk, network I/O, jumlah request/detik, latensi p95, status up/down).
2. Mengambil data berkala (pull/scrape) dari endpoint /metrics milik exporter (Node Exporter untuk host, cAdvisor untuk container) dan menyimpannya di Time-Series Database (TSDB).
3. Menghitung tren dan agregasi matematis dengan PromQL (rate, sum, avg, histogram) untuk memantau SLI/SLO, Golden Signals, dan memicu alert ambang batas.
4. Menjawab pertanyaan: "Kapan sistem mulai melambat, seberapa tinggi bebannya, dan komponen mana yang mengalami gejala?"

POLA PENGGUNAAN BERSAMA (Bab 6 + Bab 7 di Grafana):

Prometheus mendeteksi gejala lebih dulu (misal alert lonjakan HTTP 500 atau CPU 95% pada jam 10:05), lalu Fluent Bit dipakai menelusuri akar penyebabnya melalui isi log detail pada rentang waktu yang sama.

Pertanyaan analitis

1. Mengapa status container Up tidak membuktikan fungsi bisnis sehat?

   Status Up hanya berarti proses utama (PID 1) container masih berjalan. Status itu tidak menjawab apakah aplikasi benar-benar melayani request dengan benar.

Healthcheck yang hanya memeriksa availability pun belum cukup. Fungsi bisnis perlu diverifikasi lewat indikator yang lebih dalam. Readiness check yang menguji dependency, synthetic transaction, metric error rate dan latency, serta log yang menunjukkan hasil transaksi.

2. Apa perbedaan event time, waktu penerimaan collector, dan waktu penyimpanan database?

Waktu	Dicatat oleh	Keterangan
Event time	Aplikasi (producer)	Saat peristiwa benar-benar terjadi
Collector receive time	Fluent Bit	Saat record diterima atau diproses oleh collector
Database insert time	PostgreSQL	Saat row benar-benar ditulis ke tabel (misalnya DEFAULT now())

Urutannya: event time ≤ receive time ≤ insert time. Selisihnya muncul karena buffering, retry, batching (Flush 5), mode async, atau output yang sementara down. Pada konfigurasi lab, kolom time diisi dari Timestamp_Key time, yaitu waktu yang dibuat oleh collector, bukan waktu kejadian asli di aplikasi. Praktik yang baik adalah menyimpan ketiganya, memakai UTC, dan menyinkronkan jam host (NTP).

3. Mengapa nama dan email tidak dimasukkan ke structured log?

    a. Minimisasi data: hanya field yang diperlukan untuk operasi, keamanan, kepatuhan, atau analisis yang boleh disimpan.
    b. Data pribadi (PII): nama dan email dapat mengidentifikasi individu, sehingga masuk cakupan regulasi seperti UU PDP dan GDPR.
    c. Perluasan permukaan risiko: log biasanya diakses banyak pihak (developer, analyst, tim platform), dikirim ke banyak tempat, dan disimpan lama. Kebocoran log menjadi kebocoran data pribadi.
    d. Sulit dihapus: PII yang sudah tersebar di backup, ekspor, dan indeks sulit ditarik kembali saat ada permintaan penghapusan.

4. Apa risiko penggunaan request_id dari client tanpa validasi?

    a. Log injection / forging: client dapat menyisipkan karakter newline atau JSON palsu sehingga muncul entri log palsu atau struktur record rusak.

b. Pemalsuan korelasi: penyerang memakai ID milik request lain untuk mengacaukan penelusuran atau menyembunyikan jejaknya.
c. Tabrakan ID (collision): ID duplikat membuat dua transaksi berbeda tampak seperti satu.
d. Kerusakan kualitas data: panjang berlebihan membebani storage dan indeks, format tidak konsisten merusak parser dan dashboard.
e. Nilai evidence menurun: untuk audit, ID yang dikendalikan pihak luar tidak bisa dipercaya.

5. Apa yang terjadi bila buffer Fluent Bit mencapai batas 256 MB?

    a. Buffer memori penuh (Mem_Buf_Limit): input dijeda (paused) sementara. Untuk input tertentu data baru bisa hilang jika sumbernya tidak bisa menunggu.
    b. Buffer filesystem penuh (storage.max_chunks_up atau batas disk): chunk lama atau baru bisa di-drop sesuai kebijakan, atau disk bisa penuh jika tidak dibatasi.
    c. Backpressure ke Docker: jika mode logging driver blocking (tanpa fluentd-async: "true"), container producer bisa tertahan. Pada mode async, record dibuang atau di-buffer di sisi Docker dengan batas tertentu, sehingga log dapat hilang.

6. Bagaimana mendeteksi duplikasi record setelah retry?

    a. Event ID unik dari producer (UUID atau hash dari service + timestamp + sequence + isi), lalu simpan di field event_id.
    b. Unique constraint pada database, misalnya UNIQUE (event_id) dengan INSERT ... ON CONFLICT DO NOTHING agar insert menjadi idempoten.
    c. Sequence number per sumber untuk mendeteksi baik duplikasi maupun gap (record hilang).
    d. Metric retry dari Fluent Bit sebagai indikator awal kemungkinan duplikasi.

7. Kapan PostgreSQL tidak lagi sesuai sebagai backend log utama?

    a. Volume ingest tinggi dan kontinu: write amplification, WAL, dan bloat membebani database.
    b. Retensi panjang dengan data sangat besar: indeks GIN pada JSONB membengkak, query melambat, dan VACUUM serta penghapusan massal mahal.
    c. Pencarian teks bebas (full-text) skala besar atau agregasi lintas jutaan hingga miliaran record.
    d. Kebutuhan real-time seperti dashboard dan alerting dengan latency rendah.
    e. Biaya storage: penyimpanan relasional lebih mahal daripada object storage atau sistem kolumnar terkompresi.

f. Kebutuhan multi-tenant dan skala horizontal yang sulit dipenuhi satu node PostgreSQL.

8. Kontrol apa yang harus ditambahkan sebelum arsitektur serupa dipakai pada produksi?

Keamanan dan kerahasiaan

- Hapus kredensial hardcoded (labpass123), gunakan secret manager atau Docker secrets.
- Aktifkan TLS antara Docker, Fluent Bit, dan PostgreSQL, jangan expose port 24224 ke jaringan luas.
- Pisahkan credential: akun writer (INSERT saja) dan akun analyst (read-only), terapkan least privilege.
- Redaction PII dan secret di producer, diverifikasi ulang di collector.
- Audit akses query administratif dan perubahan tabel.

Reliability

- Buffer berbasis disk dengan batas jelas, kebijakan retry, dan penanganan dead-letter untuk record gagal parse.
- Replikasi, backup, dan recovery exercise untuk database serta collector (hindari single point of failure).
- Healthcheck yang menguji fungsi nyata, bukan hanya status Up.

Kualitas data

- Kontrak schema log, validasi request_id, event_id unik, timestamp UTC, dan sinkronisasi NTP.
- Versi schema dan pengujian perubahan field seperti perubahan API.